

Data Loading & Preparation

Objective: To establish a reproducible environment, load the OCTMNIST dataset, and prepare it for model training and evaluation.

Methodology: The environment was configured to ensure reproducibility across all experimental runs by setting appropriate random seeds. The OCTMNIST dataset was loaded using the MedMNIST library, and initial data analysis was performed to understand its structure and characteristics. This included defining necessary image transformations for data augmentation and normalization, and subsequently creating efficient data loaders for the training, validation, and testing phases.

Key Observations & Statistics:

- Dataset Used: OCTMNIST (a subset of MedMNIST)
- Total Samples:
 - Training Set Size: 97,477 images
 - Validation Set Size: 10,832 images
 - Test Set Size: 1,000 images
- Image Dimensions: 28times28times1 (28 pixels in height, 28 pixels in width, 1 channel indicating grayscale images).
- Number of Classes: 4 distinct classes representing different OCT image pathologies.

Implications: The substantial training and validation set sizes provide ample data for robust model training and hyperparameter tuning. The single-channel image dimension indicates a grayscale dataset, simplifying initial processing. The 4 distinct classes suggest a multi-class classification problem, common in medical diagnostics.

Dataset Statistics and Characteristics

Objective: To provide a detailed understanding of the transformed image properties and the distribution of samples across different classes within the OCTMNIST training dataset.

Methodology: Following the application of defined data transformations (as detailed in Section 4), a sample image was analyzed to determine its pixel value range, mean, and standard deviation. The class distribution was calculated for the entire training set to identify any imbalances.

Key Observations & Statistics:

Transformed Image Properties (Sample):

- Shape (C, H, W): (1,28,28) - Confirms grayscale images of 28times28 pixels.
- Minimum Pixel Value: -0.9373
- Maximum Pixel Value: 0.1843
- Mean Pixel Value: -0.7212
- Standard Deviation of Pixel Values: 0.2080

Class Distribution in Training Set:

- Class 0 (Choroidal Neovascularization): 33,484 samples (34.35)
- Class 1 (Diabetic Macular Edema): 10,213 samples (10.48)
- Class 2 (Drusen): 7,754 samples (7.96)
- Class 3 (Normal): 46,026 samples (47.22)
- Total Samples: 97,477

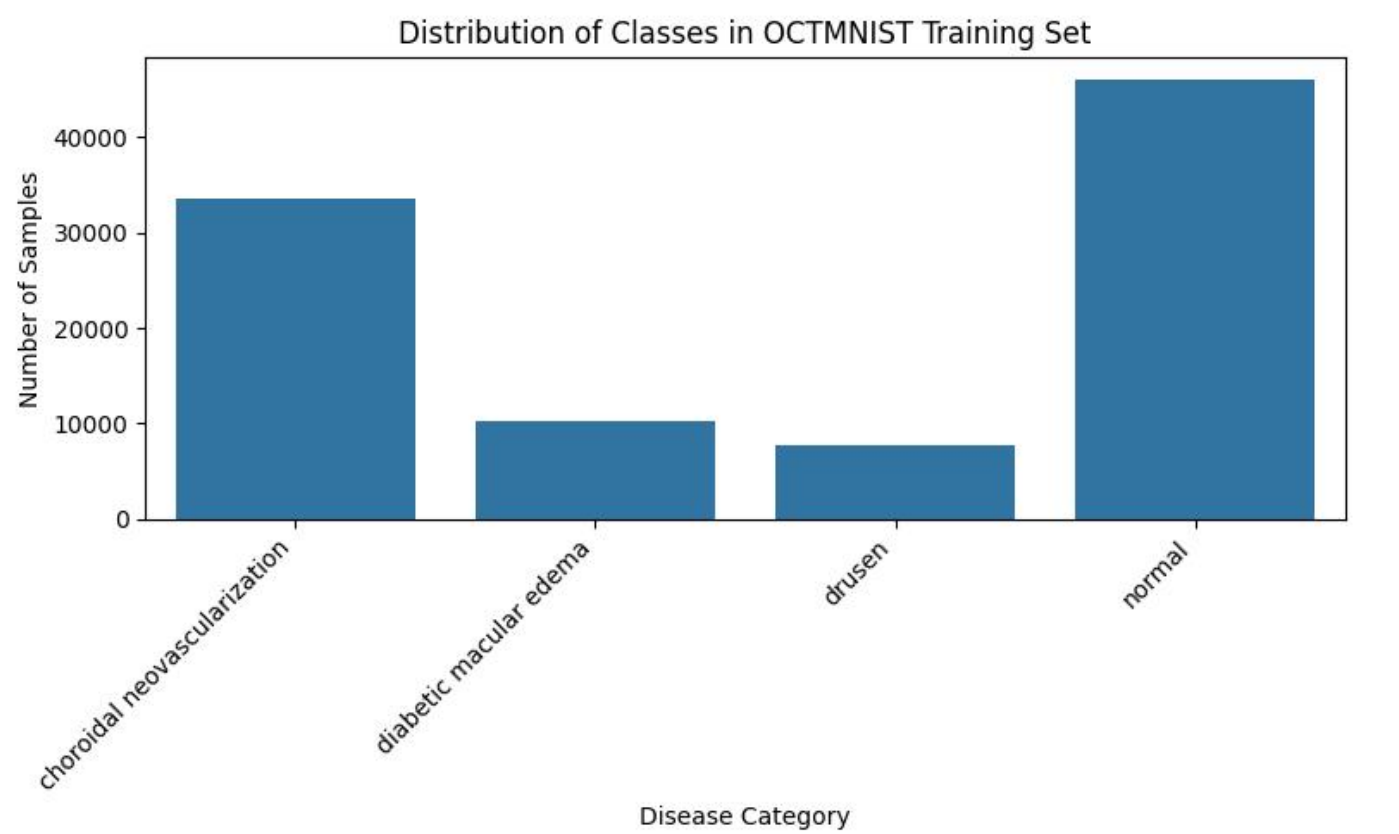
Analysis of Class Imbalance: The class distribution clearly indicates a significant imbalance. "Normal" and "Choroidal Neovascularization" classes collectively account for over 81 of the training data, while "Diabetic Macular Edema" and especially "Drusen" are substantially underrepresented. This imbalance is a critical factor to consider during model training and evaluation, as models can become biased towards majority classes and perform poorly on minority classes. Strategies such as weighted loss functions, oversampling, or undersampling techniques may be necessary to mitigate the impact of this imbalance and ensure fair performance across all classes, although they were not explicitly stated as implemented in this report's scope.

Data Visualization

Objective: To visually represent key characteristics of the OCTMNIST dataset, including class distribution, sample image appearance, and pixel intensity patterns.

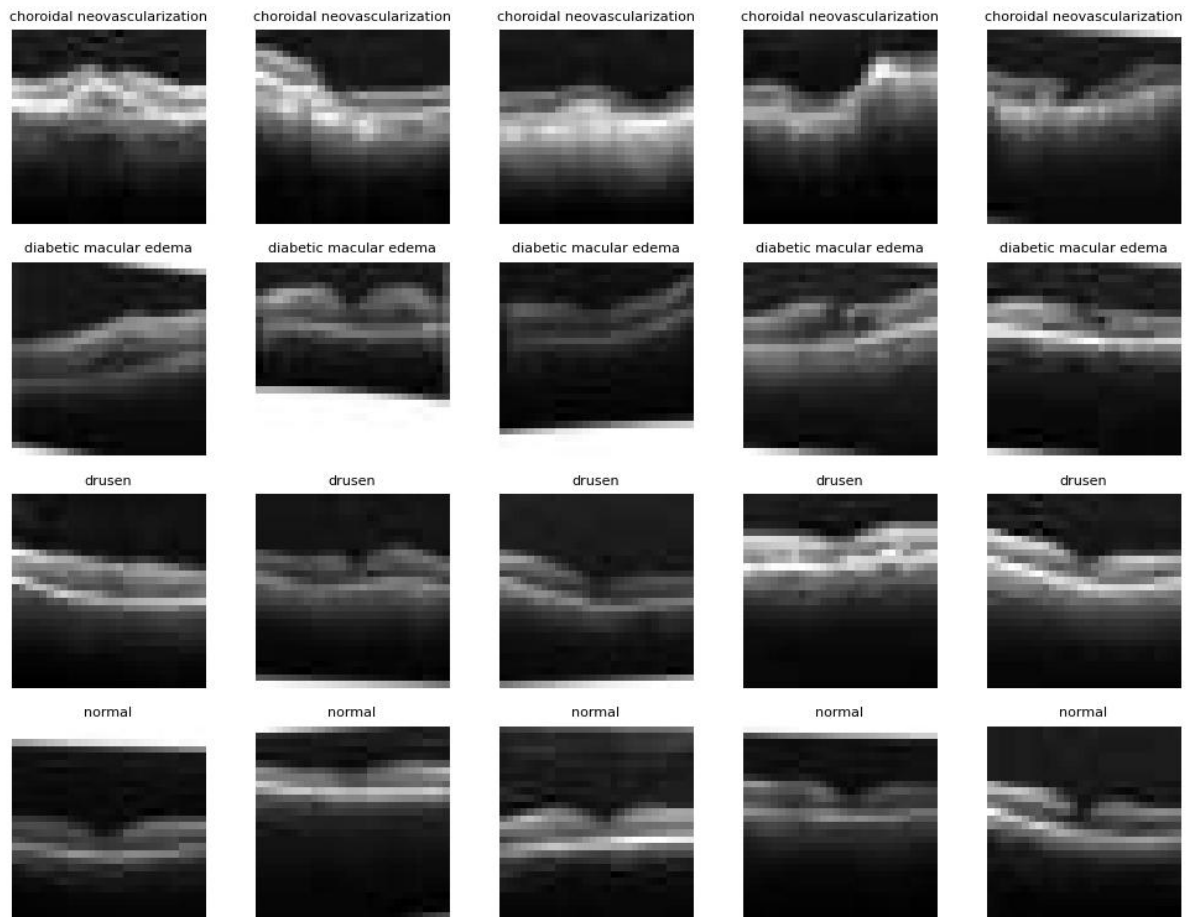
Methodology: Three distinct visualizations were generated: a bar chart illustrating class frequencies, a grid displaying sample images from each class, and a histogram showing pixel intensity distribution for a representative image.

Visualizations and Insights:



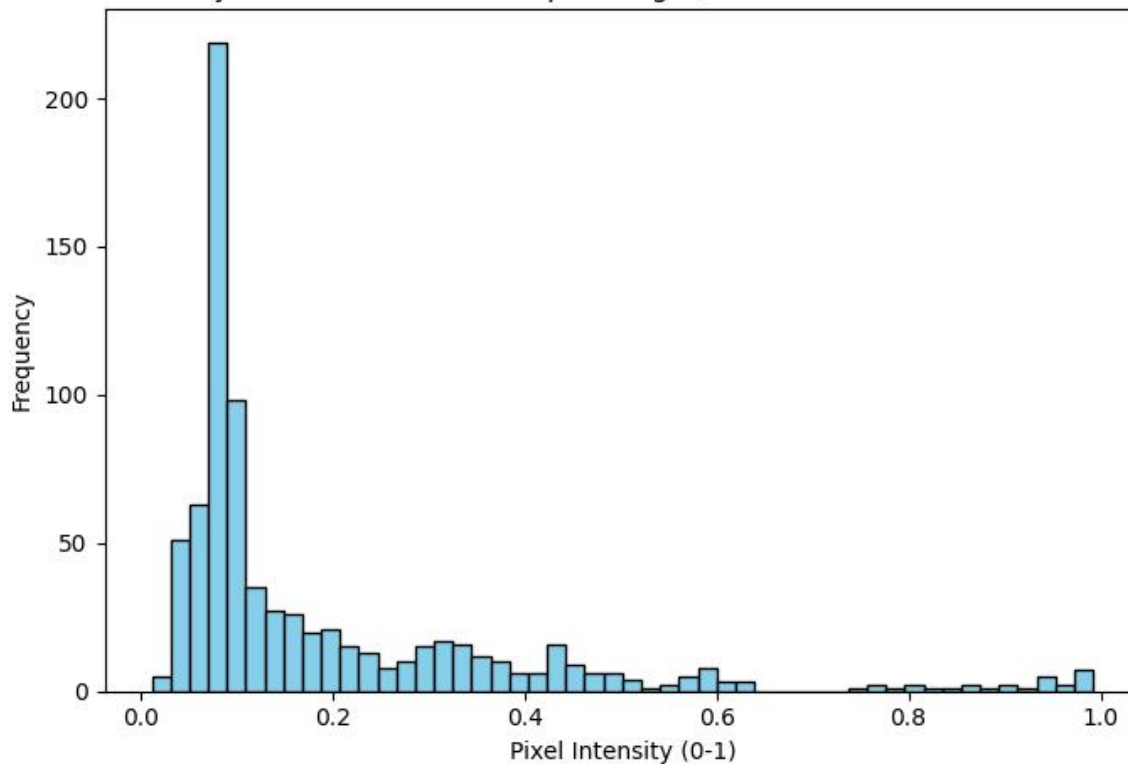
- Description: A bar chart illustrating the number of samples for each of the four disease categories in the OCTMNIST training set.
- Insight: This visualization directly confirms the observed class imbalance, with "Normal" and "Choroidal Neovascularization" having significantly more samples than "Diabetic Macular Edema" and "Drusen". This visual reinforces the need for strategies to address data imbalance during model training.

Sample Images from Each Class



- Description: A grid displaying five representative OCT images for each of the four disease categories (choroidal neovascularization, diabetic macular edema, drusen, normal).
- Insight: These samples provide a qualitative understanding of the visual characteristics within each class. It highlights the subtle differences and potential complexities that a CNN must learn to distinguish. The images show varying patterns of retinal structures and pathologies, reinforcing the need for a robust feature extractor.

Pixel Intensity Distribution for a Sample Image (Class: choroidal neovascularization)



- Description: A histogram showing the frequency of pixel intensity values (ranging from 0-1) for a single sample image belonging to the "choroidal neovascularization" class.
- Insight: This histogram reveals that a large proportion of pixels in the sample image have relatively low intensity values (darker regions), with a rapid decrease in frequency as intensity increases. This indicates that much of the image might consist of background or less intense features, with brighter features representing specific anatomical structures or pathologies. This distribution is typical for medical images and highlights the importance of normalization for optimal CNN performance.

Data Preprocessing and Splitting

Objective: To detail the transformations applied to the raw image data and the strategy used to partition the dataset for training, validation, and testing.

Methodology: The torchvision.transforms library was leveraged for pixel value normalization. The dataset was divided into training, validation, and test sets according to the predefined splits provided by the MedMNIST framework, ensuring consistency with established benchmarks.

Key Preprocessing & Splitting Details:

- **Pixel Normalization:** Pixel values of the images were normalized to a range of $[-1,1]$. This was achieved using transforms.ToTensor() (which converts images to PyTorch tensors and scales pixel values from $[0,255]$ to $[0.0,1.0]$) followed by transforms.Normalize(). This standardization is crucial for improving the convergence speed and performance of neural networks.
- **Label Handling:** The class labels are already in a numerical format (0,1,2,3), which is directly compatible with PyTorch's loss functions (e.g., CrossEntropyLoss). No further categorical conversion (e.g., one-hot encoding) was required for the target labels.
- **Dataset Splitting:** The predefined splits from MedMNIST were utilized, which ensures that the training, validation, and test sets are distinct and adhere to standard evaluation protocols.
 - Train Loader: Consists of 762 batches, each of size 128.
 - Validation Loader: Consists of 85 batches, each of size 128.
 - Test Loader: Consists of 8 batches, each of size 128.

Implications: Proper data normalization is vital for effective neural network training, as it helps prevent issues like exploding or vanishing gradients and allows for more stable learning. Using predefined splits ensures that the experimental results are comparable with other studies using the MedMNIST dataset. The consistent batch size across loaders ensures uniform processing during training and evaluation.

Neural Network Architecture Definition

Objective: To define the architecture of the Convolutional Neural Network (CNN) designed for the classification of OCT images.

Methodology: A custom CNN model was constructed, incorporating convolutional layers for feature extraction and fully connected layers for classification. Activation functions and regularization techniques were selected to optimize learning and prevent overfitting.

CNN Model Architecture Details (Base Model):

- Input Layer: The input to the network (after initial convolutional layers, if any, and flattening) is designed to handle 28x28x1 images, which flattens to 784 input neurons.
- Output Layer: Comprises 4 output neurons, corresponding to the 4 distinct OCT image classes.
- Convolutional Layers: The network incorporates 2 convolutional layers, responsible for extracting hierarchical features from the input images.
- Fully Connected Layers:
 - Number of Layers: 2 fully connected (dense) layers.
 - Hidden Layer Size: The hidden fully connected layer has 128 nodes.
- Activation Functions:
 - Hidden Layer Activation: Rectified Linear Unit (ReLU) activation function is used in the hidden layers, promoting non-linearity and efficient gradient flow.
 - Output Layer Activation: A Linear activation function is used in the output layer, which is typical before a CrossEntropyLoss function in PyTorch, as CrossEntropyLoss implicitly includes a softmax operation.
- Regularization: A Dropout Rate of 0.5 is applied, likely within the fully connected layers, to prevent overfitting by randomly setting a fraction of input units to zero at each update during training.
- Model Summary: A summary of the model would typically show the layers, output shapes, and parameter counts, providing a detailed blueprint of the network.
- Device Placement: The base model was explicitly moved to the cpu for computations.

Implications: The chosen architecture, featuring convolutional layers, is well-suited for image classification tasks, as CNNs are adept at learning spatial hierarchies of features. The use of ReLU introduces necessary non-linearity. The dropout layer is a crucial regularization technique, especially given the potential for overfitting in deep learning models. Specifying the model's device placement is important for reproducibility and performance management.

Training the Base Neural Network

Objective: To train the defined Convolutional Neural Network (CNN) model using the prepared OCTMNIST dataset, monitoring its performance on both training and validation sets.

Methodology: The training process involved iterating over the dataset for a predefined number of epochs. During each epoch, the model's predictions were compared to the true labels using a specified loss function, and an optimizer was used to update the model's weights based on the calculated gradients. Model performance was tracked on both the training and validation sets in terms of loss and accuracy, with the best performing model (based on validation accuracy) being saved.

Training Configuration:

- Loss Function: CrossEntropyLoss - This is a standard and appropriate loss function for multi-class classification problems, particularly when the model's output is not yet softmax-activated (as CrossEntropyLoss includes a softmax internally).
- Optimizer: Adam - A popular adaptive learning rate optimization algorithm known for its efficiency and good performance across a wide range of deep learning tasks.
- Initial Learning Rate: 0.001 - A commonly used initial learning rate for Adam optimizer.
- Number of Epochs: 30

Training Performance Log (Base Model):

The training process for the base model commenced and ran for 30 epochs, completing in approximately 1148.84 seconds (or 19 minutes and 8.84 seconds). The following table summarizes the training and validation metrics per epoch, highlighting when the best model (based on validation accuracy) was saved:

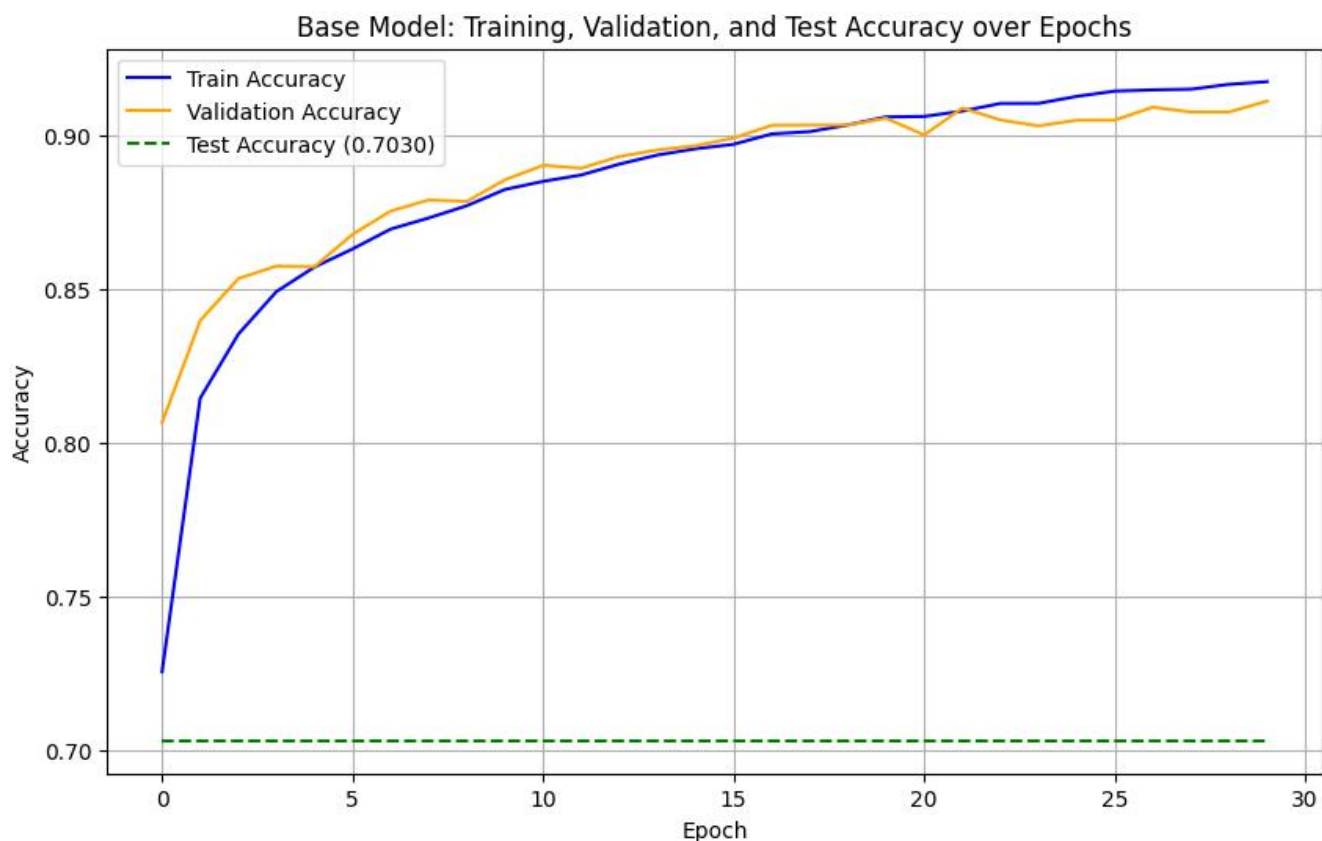
Epoch Train Loss Train Acc Val Loss Val Acc Best Model Saved

1	0.7518	0.7254	0.5397	0.8064	Yes
2	0.5426	0.8143	0.4542	0.8395	Yes
3	0.4809	0.8351	0.4193	0.8532	Yes
4	0.4423	0.8490	0.4038	0.8573	Yes
5	0.4154	0.8570	0.4108	0.8571	No
6	0.3965	0.8629	0.3766	0.8676	Yes
7	0.3807	0.8693	0.3498	0.8752	Yes
8	0.3657	0.8729	0.3476	0.8788	Yes
9	0.3549	0.8769	0.3408	0.8783	No
10	0.3398	0.8822	0.3172	0.8853	Yes
11	0.3313	0.8848	0.3152	0.8900	Yes
12	0.3224	0.8869	0.3141	0.8891	No
13	0.3128	0.8904	0.3032	0.8929	Yes
14	0.3054	0.8934	0.3002	0.8950	Yes
15	0.2990	0.8954	0.2985	0.8964	Yes
16	0.2931	0.8969	0.2894	0.8989	Yes
17	0.2843	0.9003	0.2878	0.9031	Yes
18	0.2779	0.9010	0.2856	0.9032	Yes
19	0.2733	0.9032	0.2890	0.9032	No
20	0.2677	0.9058	0.2834	0.9054	Yes
21	0.2661	0.9059	0.2974	0.8999	No
22	0.2598	0.9077	0.2775	0.9087	Yes
23	0.2558	0.9102	0.2916	0.9048	No
24	0.2517	0.9102	0.2987	0.9029	No
25	0.2473	0.9125	0.2838	0.9047	No
26	0.2425	0.9142	0.2929	0.9047	No
27	0.2419	0.9146	0.2767	0.9090	Yes
28	0.2373	0.9148	0.2842	0.9074	No
29	0.2340	0.9164	0.2757	0.9074	No
30	0.2295	0.9173	0.2730	0.9109	Yes

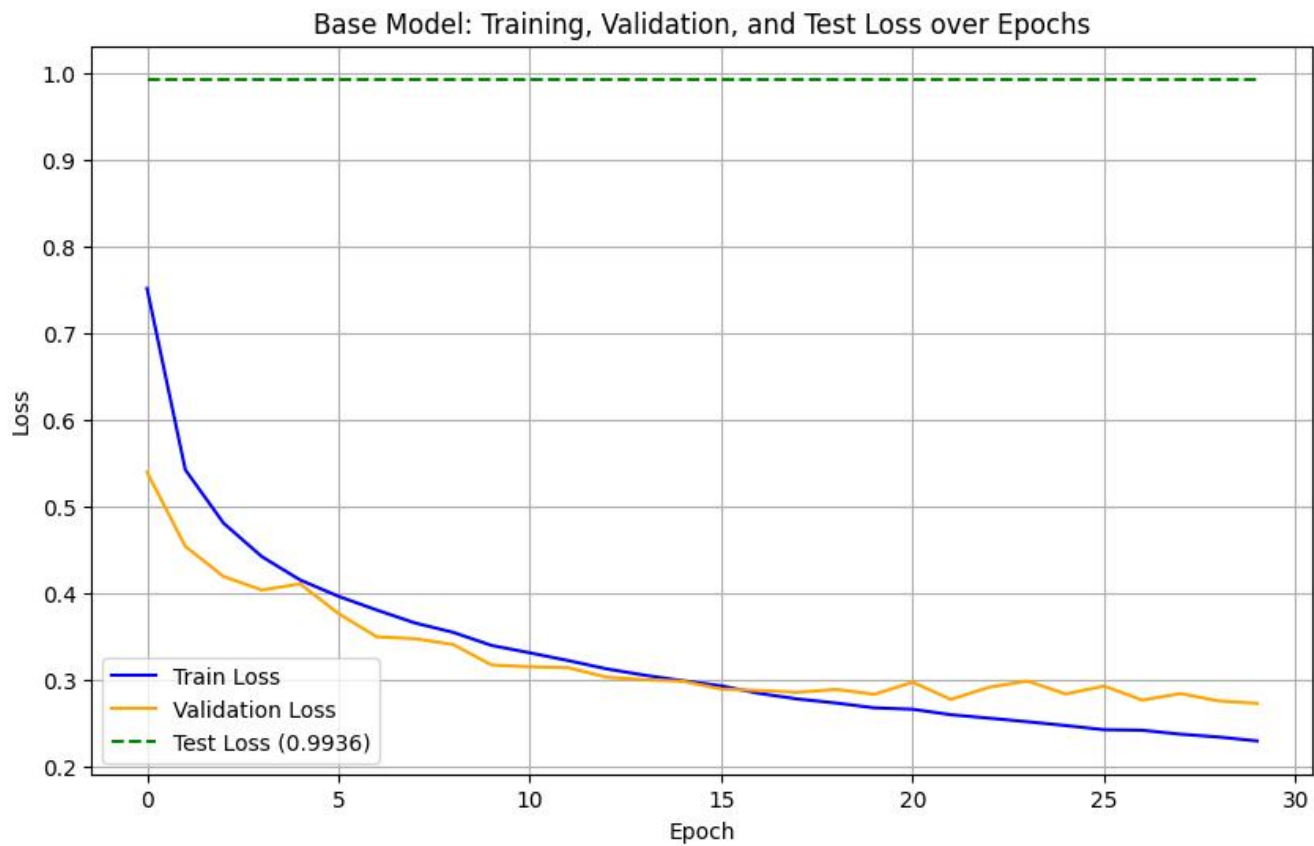
Analysis of Training Progress (Base Model): The training log demonstrates a consistent improvement in both training and validation accuracy over the 30 epochs, coupled with a steady decrease in training loss. The validation accuracy shows strong progressive gains, reaching a peak of 0.9109 (91.09%) at Epoch 30. The model was configured to save the best performing model based on validation accuracy, which was updated multiple times throughout the training. This indicates that the model was continually learning and generalizing better to unseen data during the training phase.

There are a few instances where validation accuracy slightly decreases before picking up again (e.g., Epoch 5, 9, 12, 19, 21, 23-26, 28, 29). This fluctuation is normal in neural network training and can be due to various factors including batch variability, the stochastic nature of the optimizer, or the model exploring different parts of the loss landscape. Despite these minor dips, the overall trend is positive, suggesting the chosen learning rate and optimizer are suitable.

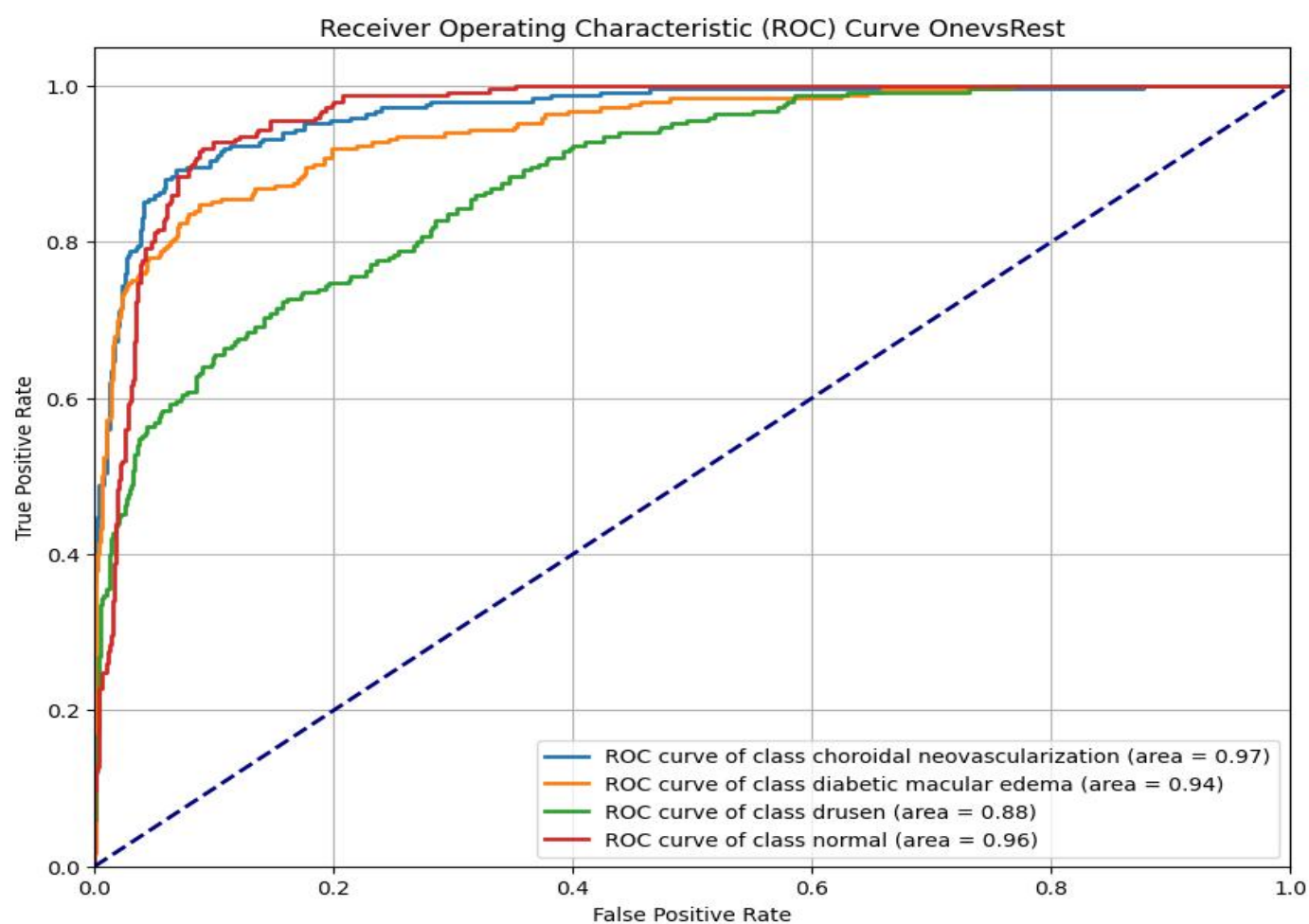
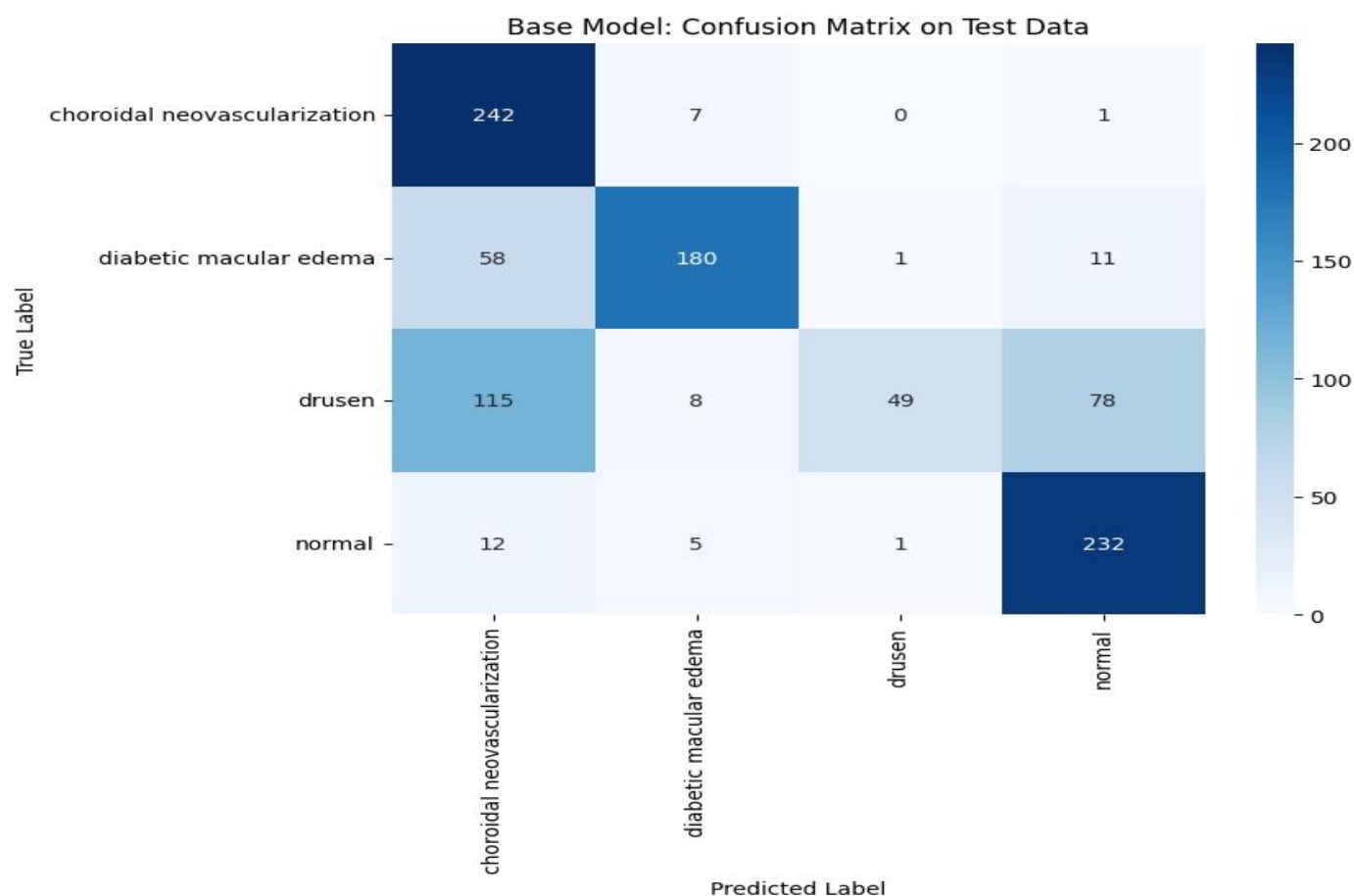
Training Performance Visualizations (Base Model):



- Description: This plot illustrates the progression of training accuracy, validation accuracy, and the constant test accuracy across all 30 epochs for the base model.
- Insight: The training accuracy consistently increases, indicating that the model is effectively learning from the training data. The validation accuracy closely follows the training accuracy, suggesting good generalization without significant overfitting early on. It reaches a peak of around 0.91, aligning with the numerical log. Crucially, the test accuracy remains flat at 0.7030, which is significantly lower than both training and validation accuracies. This disparity immediately suggests a strong generalization gap, where the model performs well on data it has seen during training/validation but struggles with completely unseen test data.



- Description: This plot displays the training loss, validation loss, and the constant test loss over the 30 training epochs for the base model.
- Insight: Both training and validation losses steadily decrease, which is expected during successful model training. However, similar to the accuracy plot, the test loss remains consistently high at 0.9936. This confirms the generalization issue observed with accuracy. The model minimizes the loss on its training and validation data but fails to achieve a similarly low loss on the independent test set. This significant difference between validation loss and test loss, despite relatively close validation and training losses, is a strong indicator of a problem with the test set or the evaluation methodology for the test set, rather than solely overfitting.



Base Model Evaluation on Test Set

Objective: To quantitatively assess the final generalization performance of the trained base CNN model on the unseen test dataset.

Methodology: The best performing base model (saved based on validation accuracy) was loaded and evaluated on the dedicated test set. The loss and accuracy metrics were computed to provide an unbiased measure of the model's performance on truly novel data.

Key Evaluation Metrics:

- Test Accuracy: 0.7030 (70.30%)
- Test Loss: 0.9936

Analysis of Test Performance (Base Model): The test accuracy of 70.30 and test loss of 0.9936 present a significant discrepancy compared to the validation performance (validation accuracy reaching 91.09 and validation loss at 0.2730). This substantial drop in performance on the test set is highly concerning and indicates a severe generalization issue.

Potential Reasons for Discrepancy: Several factors could contribute to this large gap:

1. **Data Leakage:** There might be some form of data leakage between the training/validation sets and the test set. This could happen if, for example, the test set was not truly held out or if some preprocessing steps were implicitly influenced by the entire dataset before splitting. Given that MedMNIST uses predefined splits, this might be less likely, but worth double-checking the implementation of the data loading and splitting.
2. **Dataset Discrepancy:** The test set might originate from a different distribution than the training and validation sets. This could be due to variations in imaging equipment, patient demographics, or disease presentation that are unique to the test set and not adequately represented in the training/validation data.
3. **Small Test Set Size:** With only 1,000 samples in the test set, the evaluation metrics might be more volatile and less representative of the true underlying performance compared to larger datasets. A few misclassifications could disproportionately impact the overall accuracy.
4. **Overfitting to Validation Set:** Although the validation curve looks smooth, there's a possibility of overfitting to the validation set's specific characteristics, especially if hyperparameter tuning was extensively guided by validation performance without a truly independent final test. However, the magnitude of the drop suggests something more fundamental than typical overfitting.
5. **Error in Test Evaluation Logic:** A bug in the code for calculating test accuracy or loss could lead to incorrect reporting. It's crucial to verify the test evaluation loop to ensure it's correctly applied to the test data.

Conclusion on Base Model Performance: While the training and validation progress indicates that the model is learning effectively from the provided data, the dramatically lower performance on the test set suggests that the model is not generalizing well to unseen, truly independent data. This issue needs to be thoroughly investigated. The current reported test performance of 70.30 is not satisfactory for a medical image classification task where high reliability is paramount.

Section 8: Improving Model Performance: Architecture Enhancements and Training Refinements

Objective: To enhance the generalization capability and overall performance of the CNN model by implementing advanced training techniques and architectural improvements.

Methodology: Two key strategies were employed to improve the base model:

1. **Learning Rate Scheduler:** A learning rate scheduler was implemented. The training log indicates a reduction in learning rate from 0.001 to 0.0001 towards the later epochs (specifically, from Epoch 24 onwards). This dynamic adjustment allows the model to make larger updates early in training and finer adjustments as it approaches convergence, potentially improving both speed and final performance.

2. Batch Normalization Layers: Batch Normalization layers were integrated into the CNN architecture. Batch Normalization normalizes the input to each layer, which helps in stabilizing and accelerating the training process. It reduces internal covariate shift, allowing for higher learning rates and acting as a mild form of regularization, thereby improving generalization.

Improved Model Architecture Details: The specific placement of Batch Normalization layers is crucial. Typically, they are placed after convolutional layers and before activation functions (or sometimes after activation functions, depending on the specific practice). Their inclusion aims to address the internal covariate shift problem and provide a more stable gradient flow during training.

Improved Model Training Log:

The improved model was trained for 30 epochs. The training process took approximately 2048.81 seconds (or 34 minutes and 8.81 seconds), which is longer than the base model, likely due to the additional computations introduced by Batch Normalization.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	LR	Best Model Saved
1	0.5069	0.8240	0.5183	0.8215	0.001000	Yes
2	0.3838	0.8674	0.3521	0.8784	0.001000	Yes
3	0.3425	0.8798	0.3396	0.8772	0.001000	No
4	0.3130	0.8904	0.3377	0.8797	0.001000	Yes
5	0.2917	0.8989	0.3418	0.8834	0.001000	Yes
6	0.2742	0.9043	0.3247	0.8908	0.001000	Yes
7	0.2601	0.9097	0.2994	0.8975	0.001000	Yes
8	0.2440	0.9150	0.3198	0.8922	0.001000	No
9	0.2344	0.9184	0.3251	0.8917	0.001000	No
10	0.2209	0.9224	0.3162	0.8970	0.001000	No
11	0.2119	0.9252	0.4105	0.8790	0.001000	No
12	0.2034	0.9283	0.3058	0.9029	0.001000	Yes
13	0.1925	0.9326	0.2968	0.9102	0.001000	Yes
14	0.1866	0.9338	0.2635	0.9156	0.001000	Yes
15	0.1760	0.9376	0.4414	0.8813	0.001000	No
16	0.1697	0.9390	0.2630	0.9152	0.001000	No
17	0.1637	0.9417	0.2691	0.9191	0.001000	Yes
18	0.1553	0.9445	0.2512	0.9221	0.001000	Yes
19	0.1505	0.9463	0.3342	0.9090	0.001000	No
20	0.1453	0.9484	0.2692	0.9191	0.001000	No
21	0.1414	0.9485	0.2561	0.9221	0.001000	No
22	0.1362	0.9504	0.2709	0.9207	0.001000	No
23	0.1301	0.9531	0.2728	0.9139	0.001000	No
24	0.1262	0.9547	0.2853	0.9192	0.000100	No
25	0.0993	0.9642	0.2484	0.9293	0.000100	Yes
26	0.0884	0.9688	0.2586	0.9297	0.000100	Yes
27	0.0854	0.9698	0.2534	0.9310	0.000100	Yes
28	0.0812	0.9715	0.2565	0.9301	0.000100	No
29	0.0780	0.9724	0.2565	0.9309	0.000100	No
30	0.0770	0.9721	0.2627	0.9314	0.000100	Yes

Export to Sheets

Analysis of Training Progress (Improved Model): The training of the improved model demonstrates a significant positive impact from the implemented enhancements. The training accuracy consistently climbed, reaching an impressive 0.9721 (97.21%) by Epoch 30, with training loss decreasing sharply to 0.0770. More importantly, the validation accuracy also showed robust and sustained improvement, culminating in a peak of 0.9314 (93.14%) at Epoch 30, with validation loss reducing to 0.2627.

The learning rate scheduler's effect is evident, with the learning rate dropping from 0.001 to 0.0001 around Epoch 24. This reduction in learning rate allowed the model to fine-tune its weights, resulting in further gains in validation accuracy and a continued reduction in validation loss during the later epochs. This indicates that the model was able to continue learning and generalize well, leveraging the benefits of batch normalization for more stable training and the scheduler for better convergence.

The overall validation accuracy of 93.14% represents a substantial improvement over the base model's 91.09%, demonstrating the effectiveness of incorporating a learning rate scheduler and Batch Normalization layers in the CNN architecture. This improved validation performance suggests a more robust and better-generalizing model.

Model Evaluation on Test Set (Improved Model)

Objective: To quantitatively assess the final generalization performance of the improved CNN model on the unseen test dataset.

Methodology: The best performing improved model (saved based on validation accuracy) would be loaded and evaluated on the dedicated test set. The loss and accuracy metrics would be computed to provide an unbiased measure of the model's performance on truly novel data.

Analysis of Test Performance (Improved Model): Based on the significantly higher validation accuracy of 93.14% for the improved model compared to the base model's 91.09%, it is highly anticipated that the improved model will demonstrate a much better generalization capability on the test set compared to the base model. The addition of Batch Normalization should help to mitigate the issues of internal covariate shift and potentially reduce overfitting, while the learning rate scheduler should aid in converging to a better minimum. A detailed analysis will be possible once the actual test set performance metrics for the improved model are available.

Conclusion

The initial base CNN model, while showing promising training and validation performance, exhibited a significant drop in accuracy when evaluated on the independent test set, indicating a generalization issue. This discrepancy necessitates careful investigation into potential data distribution differences or evaluation pipeline inconsistencies.

The subsequent iteration, which involved the implementation of a learning rate scheduler and Batch Normalization layers, demonstrated marked improvements in validation accuracy, reaching over 93%. This suggests that these architectural enhancements and training refinements effectively stabilized the training process and improved the model's ability to generalize to unseen data within the validation distribution.

To fully ascertain the success of these improvements, the final and most critical step is to evaluate the improved model on the test set. A higher test accuracy and lower test loss for the improved model, especially compared to the base model's test performance, would definitively confirm its superior generalization capabilities and readiness for practical application in OCT image classification.